7
Become a Spring AI Power User

In *Chapters 2* and *3*, "*Understanding Spring AI*", we covered features and functionality offered by Spring AI's APIs. We learned how to use the APIs to perform chat, content generation, RAG, function execution and much more.

Now, you are ready to dive into Spring AI's non-user-facing functionality as we cover metadata from AI interactions, observability and testing. This chapter will help frame your thinking around what it costs to solve problems using AI.

In particular, you will learn how to monitor and tune the costs of AI model interactions using metadata returned by AI providers along with metrics collected by your Spring AI applications. This will help reduce runtime costs and maximize the effectiveness of your AI model interactions, leading to better results.

As a bonus, I also developed extensions and tools to further enhance the use of Spring AI. Ideas for these extensions and tools came from the examples presented throughout this book.

In this chapter, you will learn:

- Using Chat and Model Options
- Metadata
- Observability
- Testing
- Extensions
- Tools

Technical requirements

All examples are based on Java 17 and Spring AI 1.0. Additionally, all Spring AI applications are bootstrapped with Spring Boot 3.5.x.

While you always have a choice of AI models, depending on use case and requirements, most examples give you the option of running locally using Ollama. To run the examples using Ollama, download Ollama from and installit on your system of choice.

When using Ollama, you will need to install an AI model from For example, install Meta's llama3.1 AI model using:

```
$ ollama run llama3.1
```

See the llama3.1 page for more details.

Alternatively, if you have an OpenAI or Google Cloud account, you can run the examples with OpenAI's ChatGPT or Google's Gemini.

Each example provides additional instructions on how to set up and run the Spring AI application.

Using chat and model options

We start by taking a closer look at the ModelOptions API in general since it is essential for tuning an AI model along with a model's generated response. Tuning an AI model with the options exposed by Spring AI can lead to better results.

We will not exhaustively cover all available ModelOptions since many are AI provider and model specific. Additionally, every option and its intended effect is too numerous to enumerate. However, you are encouraged to review all the options carefully as you decide on an AI model that is best suited for your application use case and requirements.

Carefully considering each option is an important factor while developing Spring AI applications since it can influence your prompts, the generated response, and even your application design.

The ModelOptions interface is not very interesting itself. It is simply a marker interface used to classify types that allow AI model interactions to be customized through different options. However, by looking at concrete subtypes, such as ChatOptions, we can understand the configurability of an AI model.

```
public interface ChatOptions extends ModelOptions {
    String getModel();
    Double getFrequencyPenalty();
    Integer getMaxTokens();
    Double getPresencePenalty();
    List<String> getStopSequences();
    Double getTemperature();
    Integer getTopK();
    Double getTopP();
    // ...
}
```

While ChatOptions are generally applicable to all chat models across all AI providers supported by Spring AI, they do vary by AI model and even by AI provider.

OpenAiChatOptions includes additional options based on the capabilities specific to ChatGPT.

```
class OpenAiChatOptions
    implements FunctionCallingOptions, ChatOptions {

    Boolean logprobs;
    Boolean parallelToolCalls;
    Integer maxCompletionTokens;
    Integer topLogprobs;
    Integer n;
    Integer seed;
    List<String> stop;
    List<FunctionTool> tools;
    Map<String, Integer> logitBias;
    ResponseFormat responseFormat;
    StreamOptions streamOptions;
    String toolChoice;
    String user;
}
```

A complete examination of all OpenAiChatOptions and their effects is beyond the scope of this book. However, if we sample a couple of options, the value is immediately apparent.

Consider "***n***". What is "***n***"? According to the Javadoc, "***n***" is:

"How many chat completion choices to generate for each input message. Note that you will be charged based on the number of tokens generated across all of the choices. Keep n at 1 to minimize costs."

That is self-explanatory. How about "**seed**"?

"This feature is in Beta. If specified, our system will make the best effort to sample deterministically, such that repeated requests with the same seed and parameters should return the same result. Determinism is not guaranteed, and you should refer to the system_fingerprint response parameter to monitor changes in the backend."

By specifying seed, you are trying to have the AI model return a similar or same response for an identical prompt.

Even similar AI models offered by different AI providers, like Microsoft and OpenAI, have differences.

Microsoft offers ChatOptions specific to Azure:

```
class AzureOpenAiChatOptions
    implements FunctionCallingOptions, ChatOptions {

    AzureChatEnhancementConfiguration enhancements;
}
```

Amazon's Bedrock Cohere chat model options differ as well:

```
class BedrockCohereChatOptions implements ChatOptions {
    ReturnLikelihoods returnLikelihoods;
    Truncate truncate;
}
```

truncate is interesting because it configures how the AI model handles inputs longer than the max token length.

Compared to Cohere, Amazon Bedrock's Jurassic2 chat model, originally from AI21, offers more robust options for Frequency, Presence Penalties, and Count Penalty. Additionally, the options include a Min `Tokens` parameter to control the minimum number of tokens the AI model should generate in response to a prompt.

```
class BedrockAi21Jurassic2ChatOptions implements ChatOptions {
    Integer minTokens;
    Penalty countPenaltyOptions;
    Penalty frequencyPenaltyOptions;
    Penalty presencePenaltyOptions;
}
```

Beyond chat, separate options exist to control the generation of Embeddings, such as the size or dimensions of the vector used to quantify the semantic meaning of content.

```
interface EmbeddingOptions extends ModelOptions {
    Integer getDimensions();
}
```

The dimensions option is the dimension of the Vector, just like X, Y, Z coordinates are dimensions in 3D Euclidean geometry. Vector dimensions depend on the AI model and can be thousands of points. Different options are provided for Function calling:

```
public interface FunctionCallingOptions extends ChatOptions {

    List<FunctionCallback> getFunctionCallbacks();

    void setFunctionCallbacks(List<FunctionCallback> l);

    Set<String> getFunctions();

    void setFunctions(Set<String> functions);

    default Boolean getProxyToolCalls() { ... }

    default void setProxyToolCalls(Boolean proxyToolCalls) { ... }

    Map<String, Object> getToolContext();

    void setToolContext(Map<String, Object> toolContext);
}
```

The List of FunctionCallbacks specifies callable functions by name, description and the JSON input type schema, defined by your application, that the AI model can call during an AI model interaction. The AI model uses the description to decide whether to call the function during the AI interaction.

Other options exist for *Streaming, Moderation, Audio Transcription, and Image Generation*.

As AI models improve, so will the options used to control the model. Keep these options in mind throughout the development of your Spring AI applications, not just when adding new features, but when enhancing existing ones. Options can change and may offer a wider degree of control. For example, context window size plays a critical role in chat memory and how much data can be sent to the AI model on request.

In any case, you should think about tuning AI model interactions using options in the same manner as you would tune your application's system performance using JVM options. Oftentimes, these will go hand in hand.

Similar to JVM's configuration settings for garbage collection (GC) algorithm and heap size, two of the more common model options used to control the variability in an AI model's generated response are Top K and Top P.

Top-K and Top-P

Top K and Top P parameters control the model's randomness as well as the quality of the generated content by limiting the possible outputs.

Top K restricts a model's output choices to the top K most probable tokens. For example, if K is 10, then an AI model will randomly select its next token from the set of tokens having the 10 highest probabilities. Lower values of K lead to more focused and deterministic output while higher values of K lead to more "creative" results.

In contrast, Top P enables the model to choose from the top set of tokens whose cumulative probability exceeds a defined threshold, or P, in a range from 0.0 to 1.0. This allows the model to adapt. For example, when P is 0.9, the model selects from the smallest set of tokens that collectively account for 90% of the probability distribution.

> TIP
>
> Refer to online resources for more thorough explanations of Top K and Top P. A rather concise and simple explanation of Top K and Top P can be found in the Medium article: https://medium.com/@8926581/understanding-top-k-and-top-p-in-prompt-engineering-00a3b93dcd40

Let's look at an example.

Example of using Top-K and Top-P

Using Meta's llama3.2 model running locally with Ollama, we will configure different values for Top K and Top P. By combining Top K and Top P, we can leverage the collective strength of both parameters by limiting the number of generated tokens (K) and further reducing the set based on cumulative probabilities (P).

The example Spring AI application appears as follows:

```
private static final int GENERATION_COUNT = 8;
private static final int SEED = 533081924;

@Bean
ApplicationRunner programRunner(ChatModel chatModel) {
  return args -> {
    int topK = parseInt(args.getNonOptionArgs().get(0));
    float topP = parseFloat(args.getNonOptionArgs().get(1));

    OllamaOptions chatOptions = OllamaOptions.builder()
      // .withSeed(SEED)
      .withTopK(topK)
      .withTopP(topP)
      .build();

    String content = "Finish the sentence, 'A long, long time ago in a...'";
    Prompt prompt = new Prompt(content, chatOptions);

    Map<String, Integer> generationCount = new HashMap<>();

    for (int count = GENERATION_COUNT; count > 0; count--) {
      String response = getContent(chatModel.call(prompt));

      generationCount.compute(response.toLowerCase(),
        (key, value) -> value == null ? 1 : ++value);
    }

    generationCount.forEach((response, count) ->
      System.out.printf("ai [%d]> %s%n", count, response));
  };
}
```

The Spring AI application requires arguments for Top K and Top P. Then, the program loops for a configured number of iterations prompting the AI model with the same directive ("*Finish the sentence, 'A long, long time ago in a...*") to determine how many times it generates the same response.

Several samples were collected by running the application with variations of K and P. The following sample output is from a single run using a Top K of 2 and a Top P of 0.96:

```
user> Finish the sentence, 'A long, long time ago in a...'?
ai [5]> ...galaxy far, far away.
ai [2]> ...galaxy far, far away. (reference to star wars)
ai [1]> ...galaxy far, far away. (from star wars(From Star Wars, of course!)
```

I think I was just insulted. In the last generation (8), the AI was clearly getting tired of me asking the same thing repeatedly.

After altering the values for K (20) and P (0.30), then running the program again, we get:

```
user> Finish the sentence, 'A long, long time ago in a...'?
ai [8]> ...galaxy far, far away.
```

All 8 generated responses were the same. For good measure, I reran this program 3 times, and in all 24 generations, across 3 runs, the result was the same.

I also experimented with the Seed. This consistently and nearly always resulted in the same generated response regardless of K and P. However, more samples should be collected using different AI models with different prompts.

Interestingly, I tweaked my prompt ever so slightly, from "Finish the sentence, 'A long, long time ago in `a...'"` to "Finish the sentence, 'A long, long time ago in'", and the AI model generated significantly different results, even when using a Top K of 2 and Top P of 0.96, as above:

```
user> Finish the sentence, 'A long, long time ago in...'
ai [1]> ...a land far, far away... (from the classic fairy tale "a long, long time ago" by disney)
ai [1]> ...a land far, far away... (from the classic disney movie "snow white and the seven dwarfs")
ai [1]> ...a far-off kingdom, hidden behind a veil of mystery and magic.
ai [2]> ...a land far, far away. (this is a reference to the classic disney movie "beauty and the beast")
ai [3]> ...a land far, far away. (reference to the classic fairy tale "sleeping beauty")
```

Yet again, this reinforces the importance of prompt and prompt engineering. It is important to remain diligent and continuously refine your prompts, especially as AI models evolve, or when adjusting model options.

Tying up

In summary, it's important to keep in mind that several factors affect the performance of a response generated by an AI model, including the model used, the data set used to train the model, parameter size, purpose and function, and how you specifically tune the model using options.

Not all AI models are equal. Size matters. For example, a 175 billion parameter model is going to perform differently than a 3-4 billion parameter model. Each has different capabilities and computational costs.

Obviously, the source, and quality of the data used to train the model is a crucial factor as is whether the model is multimodal, general purpose or specialized. Smaller models may perform specific functions and tasks on less powerful hardware efficiently with good quality while larger language models are more broadly capable, performing a variety of tasks. However, larger models will require significantly more computational resources.

In all considerations, weigh your options carefully. Pun intended.

See the complete source code for this example in the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/chat-model-options-example.

Next, we go over the metadata returned by an AI model in response to prompts and the options configured.

Metadata

This particular feature of Spring AI holds a special place in my heart since it was the first significant contribution I made to the Spring AI project back in 2023.

Like `ModelOptions`, metadata is AI provider and model specific. Different AI providers return different metadata with varying degrees of granularity.

By capturing the metadata returned by AI providers in the model's response, it gives developers valuable insights into their prompts as well as the content generated by a model. This can include total token usage and rate limits along with any content filtering applied by an AI model when processing the user's prompt.

One of the main uses for this information is to measure and track the costs of interacting with an AI model. Tokens are the currency of AI model interactions. Price per token is set by AI providers governing the models you consume in your Spring AI applications. The more tokens you use, the more it costs.

Prompts and generated content are converted to tokens. The rate of consumption is determined by how concisely you engineer your prompts in addition to the content generated and returned by the AI model in response to your prompts. Prompts do affect generation as we have already seen.

Remember in **Chapter 1**, our very first prompt stated, "In a single word, 'what is the answer to life, the universe, and everything?'" The key to this prompt was, "In a single word," which generated the response, "42".

Without beginning, "In a single word", you get a detailed, philosophical discussion on life, the Universe, everything, and a reference to Douglas Adams' book, *The Hitch-Hiker's Guide to the Galaxy, which is not what I wanted*. And, although ChatGPT and Llama3.2 both returned "42", Google Gemini returned "Forty-two", which is not the same thing. "42" is 2 tokens. "Forty-two" is 3. While that might seem insignificant, it adds up over time and with many AI interactions.

Even subtle differences using trivial examples can highlight the importance of carefully considering different AI models for the task at hand.

Of course, we learned how we can use the maxTokens property of ChatOptions to limit the generated response. However, that is not always the most effective approach, particularly if the response generated by an AI model is necessarily in depth.

For example, consider the steps necessary to sufficiently describe how to solve a math problem. How you arrive at an answer is just as important as the answer itself, especially when you are learning. Remember to always "show your work."

Still, other factors that can trigger certain interactions with an AI model include usage patterns by end users of your Spring AI application measured over a given period of time. Time is an important factor, and a limited resource. While a few users may not have much impact, thousands, hundreds of thousands, or even millions of concurrent users most certainly will. The infamous last words of, "*it works on my box*" won't survive in production.

Using the Metadata API

The two most essential components in Spring AI's Metadata API are Usage and RateLimit. Instances of these types can be acquired from ChatResponseMetadata returned in the ChatResponse generated by an AI model.

So, how much will it cost?

Let's demonstrate with an example:

```
@Bean
ApplicationRunner programRunner(ChatModel chatModel) {
   return args -> {
      String message = "Explain the theory of general relativity in a few paragraphs";
      Prompt prompt = new Prompt(message);
      ChatResponse response = chatModel.call(prompt);

      // print usage metadata
   };
}
```

After prompting the AI model and receiving the generated response, we can print invaluable metadata, starting with the Prompt and generated word counts:

```
ChatResponseMetadata metadata = response.getMetadata();
```

```
print("Prompt Word Count: %d%n",
    countWords(getContent(prompt)));

print("Generation Word Count: %d%n",
    countWords(getContent(response)));
```

Similarly, we can get the Prompt and generated token counts:

```
Usage usage = metadata.getUsage();
print("Prompt Token Count: %d%n", usage.getPromptTokens());
print("Generation Token Count: %d%n",
usage.getGenerationTokens());
```

Finally, we can observe our remaining requests and token counts in addition to our plan limits.

```
RateLimit rateLimit = metadata.getRateLimit();

print("Requests Limit: %d%n", rateLimit.getRequestsLimit());

print("Requests Remaining: %d%n",
    rateLimit.getRequestsRemaining());

print("Requests Reset (ms): %d%n",
    rateLimit.getRequestsReset().toMillis());

print("Tokens Limit: %d%n",
    rateLimit.getTokensLimit());

print("Tokens Remaining: %d%n",
    rateLimit.getTokensRemaining());

print("Tokens Reset (ms): %d%n",
    rateLimit.getTokensReset().toMillis());
```

All this information is returned by the AI provider as part of the AI model's generated response.

For this program, I used OpenAI ChatGPT (`gpt-4o`). A sample from the output is shown below:

```
user> Explain the theory of general relativity in a few paragraphs
The theory of general relativity, formulated by Albert Einstein and published in 1915, revolutionized our understanding of gravity and the fundamental st
Prompt Word Count: 10
Generation Word Count: 378
Prompt Tokens: 18
Generation Tokens: 462
Requests Limit: 500
Requests Remaining: 499
Requests Reset (ms): 120
Tokens Limit: 30000
Tokens Remaining: 29968
Tokens Reset (ms): 64
```

TIP

We could have limited the generated response returned by the AI model to 2 paragraphs simply by changing the prompt to: "Explain the theory of general relativity in 2 paragraphs," impacting the generated token count. Doing so drops the Generation Token Count from 580 down to 297.

The generated response was truncated to draw attention to the measurements. The first thing to call out from the output above is that the number of tokens is not equal to the number of words in either the prompt or the generated content.

Tokenization is a process used by an AI model to break words into tokens thereby enabling machines to understand human language.

Given we used OpenAI's gpt-4o model, we can refer to OpenAI's price chart (https://openai.com/api/pricing/) to determine the cost for both input and output tokens.

Input tokens come from prompts and media sent to an AI model. Output tokens come from generated content returned by an AI model. Token prices for input (prompts/media) and output (generated content) are different.

At the time of this writing, OpenAI charged $2.50 per 1 million tokens of input and $10.00 per 1 million tokens of output.

Using our example above, the cost for the gpt-4o AI model interaction was:

```
18 / 1,000,000 * $2.50 = $0.000018
+ 462 / 1,000,000 * $10.00 = $0.000462
—----------------------------------
$0.000480
```

Wow! Big spender.

A single AI model interaction is benign in this case. However, if your Spring AI application is serving hundreds of thousands or even millions of requests per second, this changes the price significantly. Think about all the Google searches powered by AI now.

Taking our example above to completion, if an average of 10,000 requests per second for 1 hour trigger AI model interactions, then that equates to 36 million requests per hour. Using the token counts above, that is:

```
18 * 36 * $2.50 = $1,620.00
462 * 36 * $10.00 = $166,320.00
—------------------------------
= $167,940.00
```

$167 thousand! 10K requests/second is not unreasonable for many applications, and many will be much higher. If the number of requests peaks above 100K for a sustained duration of time, then you can easily exceed several million.

Time and case matter, as do different AI models. gpt-4o-mini is cheaper and faster than gpt-4o, for instance. Along with token counts, batching, caching, and different media types, prices will vary.

Keep in mind, gpt-4o-mini is a small LLM and gpt-4o is a large, multimodal LLM, both significant differences in efficiency, capabilities, and costs. Depending on your use case and requirements, or SLAs, it may be cheaper to use a single model than 2 different models geared for different purposes. But then again, maybe not. Either way, you won't know for sure unless you measure.

An important point to highlight is that price will vary for different types of media. In my example above, we were just looking at the cost of text inputs and outputs. But, images, audio, and video are going to incur different costs, which is an important factor to consider when planning the capacity of your Spring AI applications.

Thankfully, you have a way to measure resource consumption sourced from the AI provider by using the Metadata API in Spring AI.

> Prompt optimizations, token count generation limiting and observability can go a long way in helping you to stay aware and manage your token consumption, and thereby manage your costs.

Next, we explore another way to measure token counts.

TokenCountEstimator

Using AI provider metadata is the most accurate source to view and track token usage. The AI response contains the precise token usage for both prompts and generated content supplied by the AI provider. However, it incurs an upfront cost, namely the call to the AI model itself. Fortunately, it is not the only way.

Spring AI provides the `TokenCountEstimator` interface along with the `JTokkitTokenCountEstimator` implementation out of the box.

Spring AI's `JTokkitTokenCountEstimator` is based on the `JTokkit` Java library (https://jtokkit.knuddels.de/). JTokkit is based on the `tiktoken` Python library (https://github.com/openai/tiktoken) provided by OpenAI. `Tiktoken` usage is described in more detail in the OpenAI Cookbook (https://github.com/openai/openai-cookbook/blob/main/examples/How_to_count_tokens_with_tiktoken.ipynb).

Using a simple example, we can demonstrate Spring AI's `JTokkitTokenCountEstimator` to estimate the tokens used by the last example for both the prompt and the generated content.

```
@Bean
TokenCountEstimator tokenCountEstimator(
    @Value("${examples.tokenizer.encoding-type}") String name) {

    EncodingType encodingType = EncodingType.fromName(name)
        .orElseThrow(() -> new IllegalArgumentException(
            "EncodingType [%s] not found".formatted(name)));

    return new JTokkitTokenCountEstimator(encodingType);
}

@Bean
ApplicationRunner programRunner(TokenCountEstimator tokenCountEstimator) {
    return args -> {
        Scanner scanner = new Scanner(System.in);
        String input;

        printPrompt();

        while (isNotExit(input = scanner.nextLine())) {
            if (StringUtils.hasText(input)) {
                int tokenCount = tokenCountEstimator.estimate(input);
                print("%d%n", tokenCount);
            }

            printPrompt();
        }
    };
}
```

The example uses the o200k_base encoding type, as described in the OpenAI Cookbook, under the heading "`Encodings`," which is applicable to the gpt-4o model used in the Metadata example from earlier.

The example allows prompts and generated content (after it is generated) to be fed into the program in order to estimate the token count.

The advantage of using this program is that it does not require the use of an AI model, and therefore, does not incur any costs.

When run, you will see output similar to the following:

```
prompt> Explain the theory of general relativity in a few paragraphs
11
The theory of general relativity, proposed by Albert Einstein in 1915, is a fundamental pillar of modern physics...
462
```

The JTokkitTokenCountEstimator did not quite get the prompt token count right, but it stuck the landing on the generated response like a gymnast \o/.

NOTE

One reason Spring AI's `JTokkitTokenCountEstimator was` wrong about the prompt token count is that the current implementation does not properly account for additional tokens added to the input as described in JTokkit's documentation.

Keep in mind, there is no reliable way to tell precisely what content will be generated by an AI model in response to a prompt before the prompt is sent. Therefore, it is virtually impossible to count the number of tokens in a generated response to a prompt without asking first—a classic chicken and egg problem.

The best you can do is rely on the `ChatOp` tions described earlier by using `maxTo` kens to restrict the content generated by an AI model or by using a seed to generate nearly identical content when given the same or similar prompt.

Additionally, specifying how much content an AI model should generate in the prompt, as in "*Explain the theory of general relativity in 50 words or less*," may help, but is not guaranteed.

TIP

Keep in mind that punctuation, such as periods, commas, and semicolons, along with whitespace, all add to the token count and are typically unnecessary.

See the complete source code for this example in the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/tokenizer-example.

Observability

Metadata returned by an AI provider for generated content returned in response to a prompt gives you the most accurate measurement of your AI model interactions. However, it is not the only way to measure things and gain valuable insights into the behavior of your Spring AI applications at runtime.

Spring AI integrates with Micrometer to provide you with sensible metrics out-of-the-box. The metrics include measurements for the AI model interactions in your Spring AI applications. Metadata originating from an AI provider is external, while metrics collected by your Spring AI applications using Micrometer enable you to measure and observe what is happening inside your application.

You might use a mock AI model, or Ollama running locally (akin to *Testcontainers*), to measure AI model interactions triggered by user requests to your Spring AI application. This can be useful when you want to test and tune your applications without incurring costs by interacting with an actual AI model. Using actual AI models will only consume tokens and exhaust your rate limit through long and intensive test cycles.

Given a transaction log history, you could play back user requests and determine which requests will trigger AI interactions to project the overall latency, throughput, and footprint of your application at runtime.

Whatever the case, tuning starts by collecting measurements to find precisely where bottlenecks are occurring in your application both internally and externally. Oftentimes, the simplest approach starts by looking inward first and then progressing outward.

Observing Token Count

As we learned in the last section, we can get a count of tokens used per AI model request from the AI provider directly.

Alternatively, Spring AI adds a metric, gen_ai.client.token.usage, to collect the token count for both input (prompt) and output (generated content).

We can see the value of this metric observed in a Spring AI application using Ollama, as shown below:

```
@Bean
SimpleMeterRegistry meterRegistry() {
  return new SimpleMeterRegistry();
}

@Bean
TokenCountEstimator tokenCountEstimator() {
  return new JTokkitTokenCountEstimator(
    EncodingType.O200K_BASE);
}

@Bean
ApplicationRunner programRunner(ChatClient chatClient,
  MeterRegistry meterRegistry,
  TokenCountEstimator tokenCountEstimator) {

  return readEvaluatePrintLoop((args, input) -> {

    Prompt prompt = new Prompt(input);

    ChatResponse chatResponse =
      chatClient.prompt(prompt).call().chatResponse();

    String generatedContent =
      outputAiResponse(getContent(chatResponse));

    // Metadata and Metrics
    print("%nAI Provider Token Count [%s]%n",
      getAiProviderTokenCount(chatResponse));

    print("Estimated Token Count [%s]%n",
      getEstimatedTokenCount(
        prompt,
```

```
            generatedContent,
            tokenCountEstimator));

        print("Observed Token Count [%s]%n",
          getObservedTokenCount(meterRegistry));
    });
}
```

We use a ChatClient bean to invoke AI model interactions. The TokenCountEstimator is used for comparison with metrics collected in a MeterRegistry using Micrometer.

We must declare spring-boot-starter-actuator on our application classpath to auto-configure the Spring AI application's use of Micrometer.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

While the program accesses metrics programmatically, we also explicitly enable and expose the metrics using the Web interface, like so:

```
management.endpoint.metrics.enabled=true
management.endpoints.web.exposure.include=health,metrics
```

As seen in the program above, the token count returned in the metadata by the AI provider is printed along with an estimate from the `TokenCountEstimator` and what has been observed using a Spring AI-defined Meter, `gen_ai.client.token.usage`, which is technically a Micrometer Counter.

As we have previously seen, getting the token count from the AI provider metadata is straightforward:

```
private Long getAiProviderTokenCount(ChatResponse chatResponse) {
  ChatResponseMetadata metadata = chatResponse.getMetadata();
  return metadata.getUsage().getTotalTokens();
}
```

Getting the "estimated" token count with Spring AI's `TokenCountEstimator` involves:

```
private int getEstimatedTokenCount(Prompt prompt,
  String generatedContent,
  TokenCountEstimator tokenCountEstimator) {

  String promptContent = prompt.getContents();

  int promptTokenCount =
    tokenCountEstimator.estimate(promptContent);

  int generatedContentTokenCount =
    tokenCountEstimator.estimate(generatedContent);

  int totalTokenCount =
    promptTokenCount + generatedContentTokenCount;

  return totalTokenCount;
}
```

Finally, to get the observed token count from the Micrometer Counter we do:

```
private Double getObservedTokenCount(MeterRegistry meterRegistry) {
  return meterRegistry.getMeters().stream()
    .filter(meter -> "gen_ai.client.token.usage"
```

```
            .equalsIgnoreCase(meter.getId().getName()))
        .findFirst()
        .filter(Counter.class::isInstance)
        .map(Counter.class::cast)
        .map(Counter::count)
        .orElse(0.0d);
}
```

This finds the Meter in the MeterRegistry, treats it as a Counter, and then returns the count.

When the program is run, the following output appears:

```
user> What are the main characters in Star Wars
ai> The main characters in the Star Wars saga, as presented in the original trilogy (Episodes IV-VI) and the prequel trilogy (Episodes I-III), along with the
AI Provider Token Count [864]
Estimated Token Count [706]
Observed Token Count [851.0]
```

If we continue to prompt the AI, we get:

```
user> What are the main characters in the Lord of the Rings
ai>  The "Lord of the Rings" is a high-fantasy epic written by J.R.R. Tolkien, and it has several significant characters ...
AI Provider Token Count [576]
Estimated Token Count [485]
Observed Token Count [1410.0]

user> What are the main characters in Harry Potter
ai> The main characters in the Harry Potter series by J.K. Rowling are: ...
AI Provider Token Count [759]
Estimated Token Count [629]
Observed Token Count [2155.0]
```

Compared to the token counts sourced from AI provider metadata and the TokenCountEstimator, the observed token count returned by the Micrometer Counter is cumulative.

See the complete source code for this example in the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/blob/main/observability-example/src/main/java/com/packt/spring/ai/examples/observability/TokenCountObservationApplication.java

---

Observing latency

Another important metric is latency, or the amount of time elapsed between sending a request (prompt) to an AI model and getting back a generated response. Response times vary greatly depending on load, network, and other factors, so it is an essential metric to keep in check. Perceived latency is an important factor in the responsiveness of your application and maintaining the user's attention.

We can demonstrate with a simple Spring AI application using a custom Micrometer ObservationHandler to create a Timer that measures how long the ChatModel.call(prompt) method takes.

The application accepts input from the user as the prompt to the AI model, sends a request, and repeats it until the user exits the program.

```
@Bean
ApplicationRunner programRunner(ChatClient chatClient,
    MeterRegistry meterRegistry) {

    return readEvaluatePrintLoop((args, input) -> {

        Prompt prompt = new Prompt(input);

        ChatResponse chatResponse = chatClient.prompt(prompt)
            .call()
            .chatResponse();
```

```
    outputAiResponse(chatResponse);

    TimeMeasurement timeMeasurement =
      getObservedLatency(meterRegistry);

    print("Chat Model Call:\n");
    print("Count [%d]%n", timeMeasurement.getCount());
    print("Mean Latency [%s ms]\n", timeMeasurement.getMean());
    print("Total Latency [%s ms]%n%n",
      timeMeasurement.getTotal());
  });
}
```

A `ChatClient`, `MeterRegistry` and custom `ChatModelLatencyObservationHandler` are used to prompt an AI model, time the `ChatModel.call(Prompt)` method and collect the measurement.

```
@Component
static class ChatModelCallLatencyObservationHandler
  implements ObservationHandler<ChatModelObservationContext> {

  private final MeterRegistry meterRegistry;

  @Override
  public void onStart(ChatModelObservationContext context) {
    long milliseconds = System.currentTimeMillis();
    context.put("chat.model.call.start-time", milliseconds);
  }

  @Override
  public void onStop(ChatModelObservationContext context) {

    long endTime = System.currentTimeMillis();
    long startTime = context.get("chat.model.call.start-time");
    long durationMillis = endTime - startTime;

    Timer.builder("chat.model.call.latency")
      .description("Metric measuring the latency between an AI model prompt and its generated response")
      .withTags(resolveTags(context))
      .withRegistry(getMeterRegistry())
      .record(Duration.ofMillis(durationMillis));
  }

  @Override
  public boolean supportsContext(Observation.Context context) {
    return context instanceof ChatModelObservationContext;
  }
}
```

By declaring `ChatModelCallLatencyObservationHandler` as a bean, Spring Boot will automatically register the bean with Spring Boot's auto-configured `ObservationRegistry`, as documented here: https://docs.spring.io/spring-boot/reference/actuator/observability.html.

To control the unit of time and collect timed measurements in milliseconds between AI model interactions, we abstract the use of Micrometer's Timer with a `TimeMeasurementclass`, which can be constructed from a `Timer`.

```
static class TimeMeasurement {

  static final TimeUnit DEFAULT_TIME_UNIT = TimeUnit.MILLISECONDS;

  static TimeMeasurement measured(Timer timer) {
    return TimeMeasurement.builder()
      .count(timer.count())
      .max(timer.max(DEFAULT_TIME_UNIT))
      .mean(timer.mean(DEFAULT_TIME_UNIT))
      .total(timer.totalTime(DEFAULT_TIME_UNIT))
      .build();
  }
```

```
    private double max;
    private double mean;
    private double total;
    private long count;

    // ...
}
```

A `TimeMeasurement` is built from the `Timer` obtained from Micrometer's `MeterRegistry`.

```
private TimeMeasurement getObservedLatency(MeterRegistry meterRegistry) {
    return meterRegistry.getMeters().stream()
        .filter(meter -> "chat.model.call.latency"
            .equalsIgnoreCase(meter.getId().getName()))
        .findFirst()
        .filter(Timer.class::isInstance)
        .map(Timer.class::cast)
        .map(TimeMeasurement::measured)
        .orElseGet(TimeMeasurement::zero);
}
```

The program can be run with any AI model. In the following run, the program was using Meta's llama 3.2 model with Ollama running locally.

```
user> What are the main characters in Star Wars?
...
Count [1]
Mean Latency [12196.0 ms]
Total Latency [12196.0 ms]

user> What are the main characters in Lord of the Rings?
...
Chat Model Call:
Count [2]
Mean Latency [10025.0 ms]
Total Latency [20050.0 ms]

user> What are the main characters in Harry Potter?
...
Chat Model Call:
Count [3]
Mean Latency [9751.0 ms]
Total Latency [29253.0 ms]
```

Count is the total number of time measurements taken. Total Latency is the total time spent waiting on the AI model to generate content from a series of prompts over the life of the Spring AI application. Mean Latency is the average time spent per AI model interaction.

When the Spring AI application is configured as a Servlet or Reactive-based Web application and spring-boot-starter-actuator is included on the application classpath, you can enable and expose metrics via the web. You can also access the chat.model.call.latency metric through the Spring Boot Actuator Web endpoint using your Web browser.

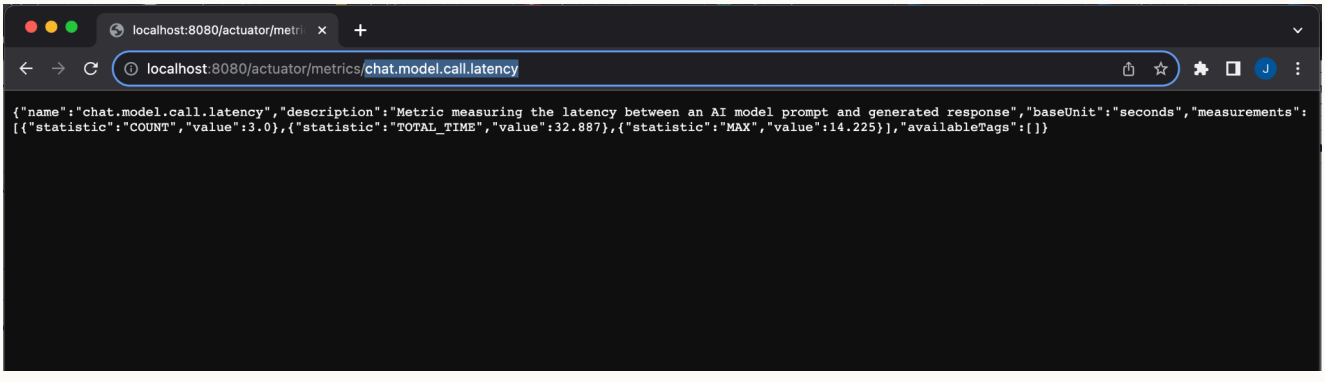


Figure 7.1: Spring Boot Actuator Metrics

Using monitoring tools, such as the Prometheus time-series database, Spring Boot Actuator metrics can be scraped to aggregate, roll up, and track metrics visually over time, as well as set alerts and perform a variety of other management activities.

See the complete source code for this example in the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/blob/main/observability-example/src/main/java/com/packt/spring/ai/examples/observability/TimedChatModelCallObservationApplication.java

---

Observing Call Count

Not unlike the example above used to monitor AI chat model call latency, we can also monitor the active call count. That is, we can monitor how many Spring AI application requests are currently making an AI chat model call and are subsequently waiting on the model to generate a response.

This can be a really useful measurement when correlated with the amount of system resources being consumed by the application at any given instant using APM tools like JDK Mission Control and Java Flight Recorder.

To do this, we use a Micrometer Gauge inside a custom `ObservationHandler`.

```
class ChatModelActiveCallCountObservationHandler
  implements ObservationHandler<ChatModelObservationContext> {

  private final AtomicLong activeCallCount = new AtomicLong(0L);
  private final MeterRegistry meterRegistry;

  public void onStart(ChatModelObservationContext observationContext) {
    Gauge.builder(METER_NAME, this::getActiveCallCount)
      .description(DESCRIPTION)
      .tags(resolveTags(observationContext))
      .register(getMeterRegistry());

    getActiveCallCount().incrementAndGet();
  }

  public void onStop(ChatModelObservationContext context) {
    getActiveCallCount().decrementAndGet();
  }

  public boolean supportsContext(Observation.Context context) {
    return context instanceof ChatModelObservationContext;
  }
}
```

This simple ObservationHandler just increments the active call count when a program Thread sends a request to an AI model, then decrements the counter when the AI model returns the generated response.

When used in a Spring Boot application that simulates a series of concurrent AI chat model interactions using Threads, we see output similar to:

```
Prompt [Describe a journey through the Milky Way galaxy]
Active Call Count [0]
Active Call Count [1]
Prompt [Generate a humorous dialog between a cat and dog]
Active Call Count [2]
Prompt [How would you invent time travel]
Active Call Count [3]
...
Active Call Count [2]
Prompt [What are the downsides of being a super hero]
Prompt [If you could have any job what would you do and why]
Active Call Count [3]
...
Active Call Count [2]
```

```
Active Call Count [2]
Active Call Count [1]
...
Active Call Count [0]
```

The Spring AI application randomly selects a prompt submitted by no more than 3 active Threads at a given moment. The number of Threads is configurable, as are the prompts sent to the AI model. During execution, we observe the Gauge to determine how many Threads are actively waiting on a generated response to a prompt sent to the AI model.

NOTE

See the complete source code for this example in the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/blob/main/observability-example/src/main/java/com/packt/spring/ai/examples/observability/GaugedChatModelCallObservationApplication.java

---

Testing

How do you test Spring AI applications? How do you test AI generated content in general?

How do you assert that the content generated by an AI model is accurate and correct? What if you don't know the answer? What sort of automated tests do you write?

Of course, the answer to all these questions depends on many factors: use case, requirements, licenses, SLAs, legal obligations, function, nature of the service, etc. Mostly, it boils down to trust and reliability.

We are continuously looking to our tools, automation, and electronic assistants to provide us with answers, to "take care of things". That is a tremendous amount of trust on our part and a significant amount of responsibility on the service provider.

It is one thing for us to ask AI a question and get a wrong answer. It is something else if an AI is used to enhance software and it gives other users the wrong answer. There is a certain liability in that.

For example, how about the heated debate of self-driving vehicles? If a self-driving vehicle gets into an accident, is it the driver's, vehicle's, software's, or manufacturer's fault?

If AI gives me the wrong driving directions to my destination, I might get lost and annoyed, but maybe the worst that happens is, I arrive late. If AI gets the dosage for a patient's medication wrong, then the consequences can be quite severe.

In all 3 cases, it is certainly a bad time for the AI to hallucinate. Without proper disclaimers, disclosures, legally binding contracts or agreements, companies are responsible, liable and can even be held accountable.

"*Better safe than sorry*," as my mom always used to say.

While there is no definitive method for testing with absolute certainty that what an AI model generates is accurate or correct all the time, there are certain safeguards that can be taken.

Use evaluation testing

One technique for testing is to use *Evaluation Testing as* summarized in Spring AI's reference documentation, https://docs.spring.io/spring-ai/reference/api/testing.html.

In a nutshell, this technique uses an AI model, which can be the same or a different model from the one used to generate the response, to evaluate the generated response from the AI model under test and determine whether the response returned is appropriate and relevant to the prompt given. The assertion in this case is usually a yes (true) or no (false) answer.

Evaluation testing is most effective when the AI model's response can be rooted in context, such as by using Retrieval Augmented Generation (RAG), providing a source of truth, or a reference against which to evaluate the generated content.

Spring AI provides a base interface for making such evaluations, the Evaluator.

```
@FunctionalInterface
interface Evaluator {
EvaluationResponse evaluate(EvaluationRequest request);
}
```

The `EvaluationRequest` provides the means to supply the original prompt, AI-generated content, and any contextually relevant information, such as any data supplied through RAG. The `EvaluationResponse` essentially returns a true or false result of the evaluation, but additionally includes a score and the ability to provide feedback on the evaluation.

However, rather than focus on what Spring AI already gives you, we offer other ideas that might prove to be invaluable in your testing efforts based on your AI use case.

Use discrete measurements

Do you remember back in **Chapter 2** when we covered the Moderation Model API?

Another useful technique when testing is to deliberately input harmful or sensitive prompts to ensure the AI model properly categorizes "bad" prompts.

In the simplest case, a test can query a boolean value and make an assertion. For example, the test can call `isHarrassement()`, `isSelfHarm()`, or any of the other boolean methods on the `Categoriesobject` to make an assertion:

`assertThat ( categories.isSelfHarm() ) .isTrue() ;`

Both positive and negative tests work in this case.

Alternatively, a test can also measure that certain bad prompts exceed a predefined threshold. For example, by calling getHarassment(), which returns a double value between 0.0d and 1.0d, we can assert that the value is greater than or equal to an acceptable threshold, such as 75%, or 0.75d:

```
assertThat(scores.getHarassment()).isGreaterThanOrEqualTo(0.75d);
```

Additionally, the metadata returned from Moderation can be used to block, filter or return alternate content from your own application's APIs. These filters or alternate content can then be tested as proper return values.

Oftentimes, the response generated by an AI model will not be returned verbatim; rather, the content will be curated, especially when accuracy is required. Spring AI's Advisors API, which can be used to intercept the generated content, can help in this regard.

Lastly, specifying a seed in `ChatOptions` may help generate more consistent content that allows a certain degree of control when testing, asserting that specific keywords are present, or that certain patterns in the generated content match a *Regular Expression*. However, keep in mind that certain options, such as seed, are only applicable to some AI models, not all.

Use pre-generated answers.

Another useful technique is to feed an AI model with a series of prompts commonly asked by users in the context of your application's domain. Then, you pre-generate answers that can be curated, stored, and returned in response to similar prompts asked by users in subsequent interactions.

The procedure works as follows:

1. First, a user submits a request containing a query or prompt via the application. The prompt is vectorized and stored in a Vector Store.
2. Next, a similarity search is run to match the user's prompt against similar prompts entered by users during previous interactions. Prompts are continuously stored by the application along with the pre-generated answer.
3. If the prompt is similar, satisfying a predefined threshold, and if an association does not already exist, then store a new association between the prompt and the stored, pre-generated answer
4. Upon completion, simply return the pre-generated answer
5. If a pre-generated answer does not already exist, decide what to do, such as allowing the AI to generate an answer in real-time or something else

In summary, if questions asked by users are similar enough, return the same, curated, quality-controlled answer.

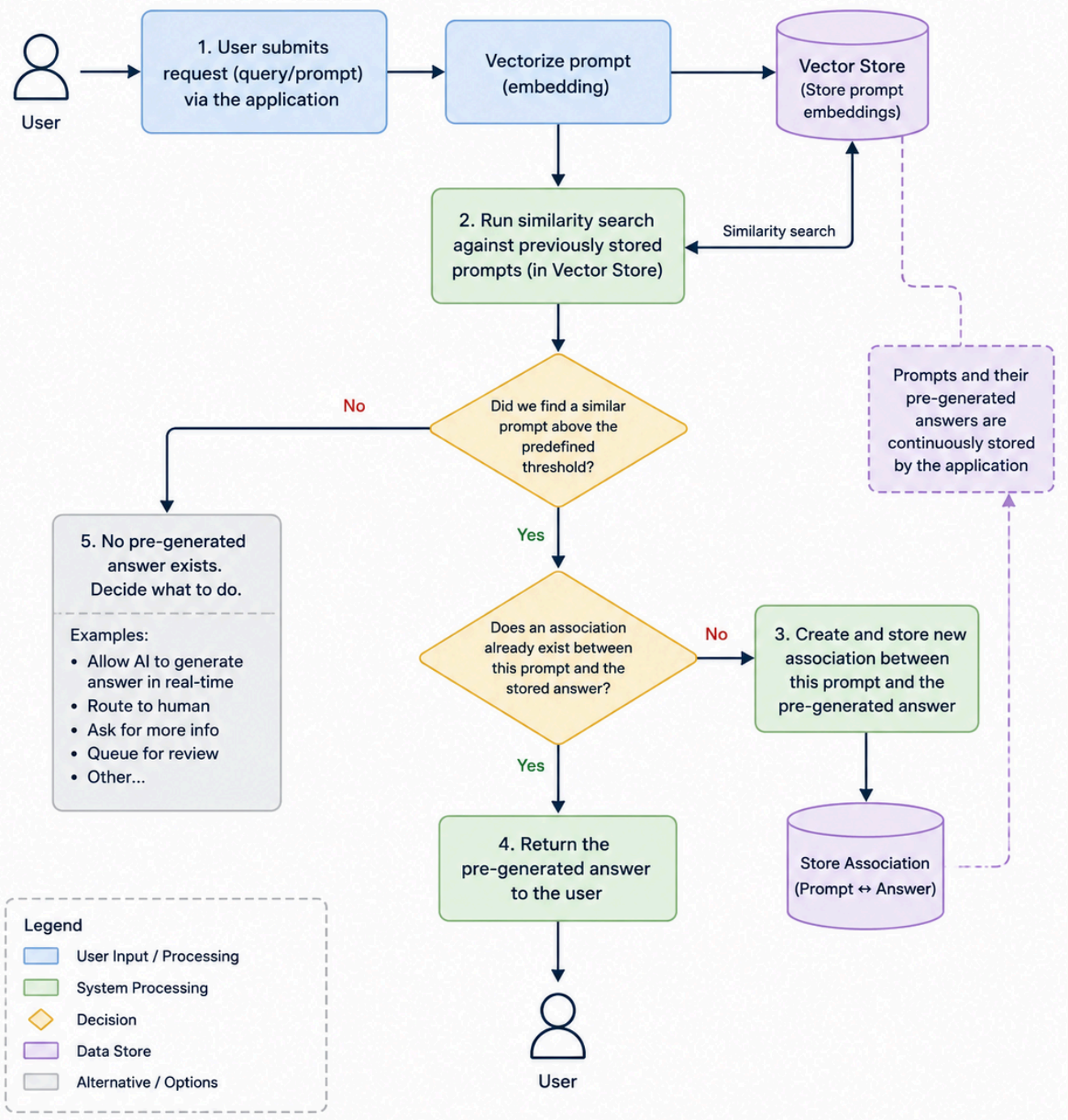


Figure 7.2: Pre-Generated Answers Workflow

By using pre-generated answers, you avoid more costly AI model interactions to generate a new answer or regenerate an answer each time a user makes a similar request, especially if the question is semantically similar. It is not uncommon for users to ask the same question twice, and maybe, only slightly differently.

From time to time, pre-generated answers can be reviewed for accuracy and relevance. Additionally, it might not be that uncommon to have multiple different explanations for the same or similar questions, especially when asked more than once.

As a result, it is much easier to test when the content does not change or is reasonably deterministic when computed from a similarity threshold. Of course, this technique is only reliable if the content generated changes infrequently and remains relatively static over time. Historical vs future weather reports are a good example. We are not quite at Back to the Future levels when it comes to predicting weather yet.

In addition to significantly reducing costs for generated and regenerated answers to similar questions, when combined with caching, this can even decrease latency and improve throughput—a win-win.

How might we go about implementing this technique? I offer an example of one possible approach. Several different approaches could be used.

In this example, we will be leveraging a `VectorStore`, *Similarity Search* and `ChatClient` APIs to implement a simple Q&A system. Such a system could be used to answer How-Tos, such as, "*How do I solve a quadratic equation?*"

First, we start by defining application domain model types for a Question and an Answer:

```
record Question(String name, Document document, Answer answer)
    implements Identifiable<String>, Nameable<String> {

    public boolean isMatch(Document document) {
        return document != null
          &&isMatch(document.getContent());
    }

    public boolean isMatch(Question question) {
        return question != null
          &&isMatch(question.get());
    }

    public boolean isMatch(String question) {
        return StringUtils.hasText(question)
      &&document().getContent().equalsIgnoreCase(question);
    }

    // ...
}
```

A Question takes a Document containing the user's question, which gets vectorized and stored in a VectorStore. The question is later used in a Similarity Search to find similar questions. A Question has a direct association to its Answer for convenience.

```
record Answer(UUID id, String content)
    implements Identifiable<UUID> {

    // ...
}
```

An `Answer` encapsulates the AI model's (pre)generated response. Several other data structures and types were used and created to simplify the implementation. They are very intuitive and non-essential to this discussion.

Next, we define a simple interface for the service used to ask questions and get answers:

```
interface AnswerService {
    Answer answer(Question question);
}
```

The default implementation is `SmartAnswerService`.

```
@Service
class SmartAnswerService implements AnswerService {

    @Override
    public Answer answer(Question question) {
        Assertions.assertQuestion(question);

        return getRepository().findBy(question)
            .map(HowTo::getAnswer)
            .orElseGet(() -> findAnswerFromSimilarQuestions(question));
    }

    // ...
}
```

You can see here that we first try to look up the stored Answer using the precise Question. If an exact match cannot be found, then SmartAnswerService proceeds to run a *Similarity Search*:

```
Answer findAnswerFromSimilarQuestions(Question question) {
  List<Document> similarDocuments =
    findSimilarDocuments(question);

  if (Utils.isNotEmpty(similarDocuments)) {
    Answer answer = findAnswerBySimilarDocuments(
      question, similarDocuments);

    if (Answer.isNotUnknown(answer)) {
      return answer;
    }
  }

  throw AnswerNotFoundException.from(question);
}

List<Document> findSimilarDocuments(Question question) {
  String query = question.get();

  SearchRequest request = SearchRequest.query(query)
    .withSimilarityThreshold(getSimilarityThreshold())
    .withTopK(getTopK());

  return getVectorStore().similaritySearch(request);
}

Answer findAnswerBySimilarDocuments(
  Question question,
  List<Document> similarDocuments) {

  return getRepository().findBy(similarDocuments)
    .map(howTo -> howTo.add(store(question)))
    .map(HowTo::getAnswer)
    .orElse(Answer.UNKNOWN);
}
```

The similarity threshold and TOP K used in the Similarity *Search can be* configured in `application.properties`. Values default to 75% and 10, respectively.

In short, similar Documents tied to individual Questions are searched in the VectorStore. Any Documents found during the search are then used to look up the corresponding Questions.

If found, the user's Question is stored along with other similar Questions, the Answer gets associated with the new Question, and the same Answer is returned. If similar Questions cannot be found, then an AnswerNotFoundException is thrown. Optionally, as a bonus, I implemented an AiEnabledSmartAnswerService to catch and handle the AnswerNotFoundException by calling a configured AI model to provide an answer in real-time. However, while an answer can be generated in real-time, it comes at the expense of increased token cost and latency.

```
class AiEnabledSmartAnswerService extends SmartAnswerService {

  @Override
  public Answer answer(Question question) {
    try {
      return super.answer(question);
    }
    catch (AnswerNotFoundException tryAgain) {
      return answerWithAi(question);
    }
  }

  Answer answerWithAi(Question question) {
    Answer answer = promptAi(question);

    Question answeredQuestion = Question.copy(question)
      .answered(answer)
      .build();
```

```
        record(answeredQuestion);

        return answer;
    }

    Answer promptAi(Question question) {
        Prompt prompt = new Prompt(question.get());

        String answer = getChatClient().prompt(prompt)
            .call()
            .content();

        return Answer.from(answer);
    }
}
```

This service can be enabled with the ai-enabled-answers Spring profile.

Of course, the focus remains on pre-generated answers so that we can reduce costs and test based on reliable content.

By way of example, the following test cases demonstrate and assert the desired outcome:

```
@Autowired
private AnswerService answerService;

@Test
void howToSolveLinearEquationReturnsAnswer() {
    Answer answer = this.answerService.answer(
        Question.from("How to solve a linear equation?"));

    assertThat(answer).isNotNull();
}

@Test
void howToSolveQuadraticEquationWithSimilarQuestionReturnsSameAnswer() {

    Answer answer = this.answerService.answer(
        Question.from("How do I solve a quadratic equation?"));

    assertThat(answer).isNotNull();

    Answer answerAgain = this.answerService.answer(
        Question.from("I am trying to solve a quadratic equation."));

    assertThat(answerAgain).isSameAsAs(answer);
}

@Test
void questionWithNoAnswer() {

    assertThatExceptionOfType(AnswerNotFoundException.class)
        .isThrownBy(() -> this.answerService.answer(
            Question.from("How do I tie my shoes?")))
        .withMessage("Answer to Question [How do I tie my shoes?] not found")
        .withNoCause();
}
```

See the complete source code for this example in the GitHub repository: https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/pre-generated-answers-example.

Next, we look at custom extensions of the Spring AI framework.

Extensions

Spring AI is a framework, and like any invaluable framework, it offers extension points to customize the functionality and behavior to fit the problem at hand.

I said it before, and it is worth repeating, Spring exemplifies the Open/Closed principle. There has been no problem that I haven't been able to solve with Spring, either out-of-the-box, or by way of extension.

Throughout the course of writing this book, I thought of many useful extensions to Spring AI that helped me simplify the implementation of the examples. Indeed, many of these custom extensions can be used more generally to help you build intelligent applications using Spring AI and Spring Boot.

All the custom extensions to Spring AI can be found in the spring-ai-extensions module of the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/spring-ai-extensions .

Enabling a ChatClient with a composable ChatModel

A pattern commonly used throughout this book, and that you will likely use when building Spring AI applications, is to define a ChatClient wrapping a ChatModel as a Spring bean.

```
@Bean
ChatClient chatClient(ChatModel chatModel) {
  return ChatClient.builder(chatModel).build();
}
```

As discussed in **Chapter 2**, using the ChatClient affords you many more capabilities, like using Advisors. While a relatively easy task and simple to generate, this is a very repetitive step, one that is easy to forget.

As part of the Spring AI extensions, I offer the `@EnableChatClient` annotation, which can be defined on your `@SpringBootConfiguration class` :

```
@SpringBootConfiguration
@EnableChatClient
static class MyApplicationConfiguration {

  // define beans
}
```

However, building a `ChatClient` from a `ChatModel` is not all that the `@EnableChatClient` annotation provides.

Additionally, the imported configuration allows you to define a Consumer bean to further customize the `ChatClient.Builder` used to build the `ChatClient` :

```
@Bean
Consumer<ChatClient.Builder> chatClientBuilderCustomizer() {
  return chatClientBuilder -> {
    // mutate the ChatClientBuilder
  };
}
```

Still, that is not all.

By default, Spring AI does not allow you to declare more than one AI provider on your application classpath at a time. If you attempt to do this, a NoUniqueBeanDefinitionException will be thrown:

```
Caused by:
 o.s.beans.factory.NoUniqueBeanDefinitionException: No qualifying bean of type 'o.s.ai.chat.model.ChatModel' available: expected single matching bean
```

Therefore, `@EnableChatClient` configuration additionally defines a primary `CompositeChatModel` bean that wraps all other `ChatModel` implementations found in your application `classpath` . This enables you to seamlessly switch between different `ChatModel` implementations at runtime.

```
@Bean
@Primary
CompositeChatModel compositeChatModel(List<ChatModel> models) {
  return CompositeChatModel.of(models);
}
```

As seen in the bean definition above, it relies on the `CompositeChatModel` class.

```
class CompositeChatModel implements Iterable<ChatModel>, ChatModel {

  private volatile ChatModel currentChatModel;
  private final Set<ChatModel> chatModels;

  Optional<ChatModel> findBy(Predicate<ChatModel> predicate) {
    return stream()
      .filter(predicate)
      .findFirst();
  }

  ChatModel requireBy(Predicate<ChatModel> predicate) {
    return findBy(predicate)
      .orElseThrow(() ->
        new ChatModelNotFoundException(
          "ChatModel could not be found with Predicate [%s]."
            .formatted(predicate)));
  }

  CompositeChatModel use(ChatModel chatModel) {
    Assert.notNull(chatModel, "ChatModel is required");

    getChatModels().add(chatModel);
    this.currentChatModel = chatModel;

    return this;
  }

  CompositeChatModel use(Class<ChatModel> type) {
    Assert.notNull(type, "Type of ChatModel is required.");

    Predicate<ChatModel> chatModelByType = type::isInstance;
    ChatModel chatModel = requireBy(chatModelByType);

    return use(chatModel);
  }

  // ...
}
```

Given a reference to the `CompositeChatModel`, it is very easy to switch the ChatModel implementation in use:

`compositeChatModel.use(OpenAiChatModel.class) ;`

This can be useful for rapid prototyping and testing efforts, as well as in production Spring AI applications needing to use different models with different capabilities for different use cases.

DecoratedSimpleVectorStore

As the name implies, `DecoratedSimpleVectorStore` uses the Decorator pattern, https://en.wikipedia.org/wiki/Decorator_pattern.

As seen throughout this book, we made use of Spring AI's `SimpleVectorStore` to store embeddings (vectors) generated by Embedding Models for various types of content. The vectors are used in Similarity Search, which can be applied during RAG.

Although Spring AI's `SimpleVectorStor` allows the contents of its in-memory storage to be written to and read from disk as JSON, it does not currently allow you to store a Document for which an embedding was already computed without recomputing the embedding for the Document's content.

For example, in the previous section on testing, I loaded pre-generated answers stored as JSON from disk. Along with the vectorized `Document`, I also stored additional data about the `Question`, such as its `Answer`. While I could have chosen a different, more normalized schema to store a `Question`, it was easier to simply de/serialize a `Question` with its `Answer` from/to JSON using Jackson, as is.

To avoid recomputing the embedding for a `Document`, it was a simple enhancement:

```
class DecoratedSimpleVectorStore extends SimpleVectorStore {

  @Override
  public void accept(List<Document> documents) {
    List<Document> embeddedDocuments = store(documents);

    documents = reduce(documents, embeddedDocuments);

    super.accept(documents);
  }

  private List<Document> reduce(List<Document> source,
    List<Document> exclude) {

    List<Document> documents = new ArrayList<>(source);

    documents.removeAll(exclude);

    return documents;
  }

  private List<Document> store(List<Document> documents) {
    return documents.stream()
      .filter(this::isEmbeddingPresent)
      .map(this::store)
      .toList();
  }

  private Document store(Document document) {
    this.store.put(document.getId(), document);
    return document;
  }
}
```

> See the complete implementation in the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/blob/main/spring-ai-extensions/src/main/java/io/codeprimate/extensions/spring/ai/vectorstore/DecoratedSimpleVectorStore.java.

Custom text splitters

When performing a similarity search, how you split up the content matters.

As we saw in the song similarity search example from **Chapter 2** - Embeddings Model API, splitting the song by refrain (a line or single lyric) or by verse, significantly improved our ability to match songs when only a few words from the lyrics of the song were given.

Spring AI provides the TokenTextSplitter out-of-the-box, which also looks for natural line breaks, such as the end of a sentence marked by punctuation [.?!\n]. However, the TokenTextSplitter does not handle all punctuation, nor does it exclude words that don't add value to the prompt, such as certain articles and conjunctions.

"*Anyway, I was about to say*" and "*The phrase is from Star Wars, of course*", are two examples of sentences containing the conjunctions "*Anyway" and "of course*". The conjunctions don't add any meaningful value to the prompt and won't change the outcome when written as "*I was about to say"* and *"The phrase is from Star Wars*."

As a result, the custom `TextSplitter` extensions offered here help by removing fluff to just focus on stuff that matters.

The custom `TextSplitters` form a hierarchy and provide common functionality.

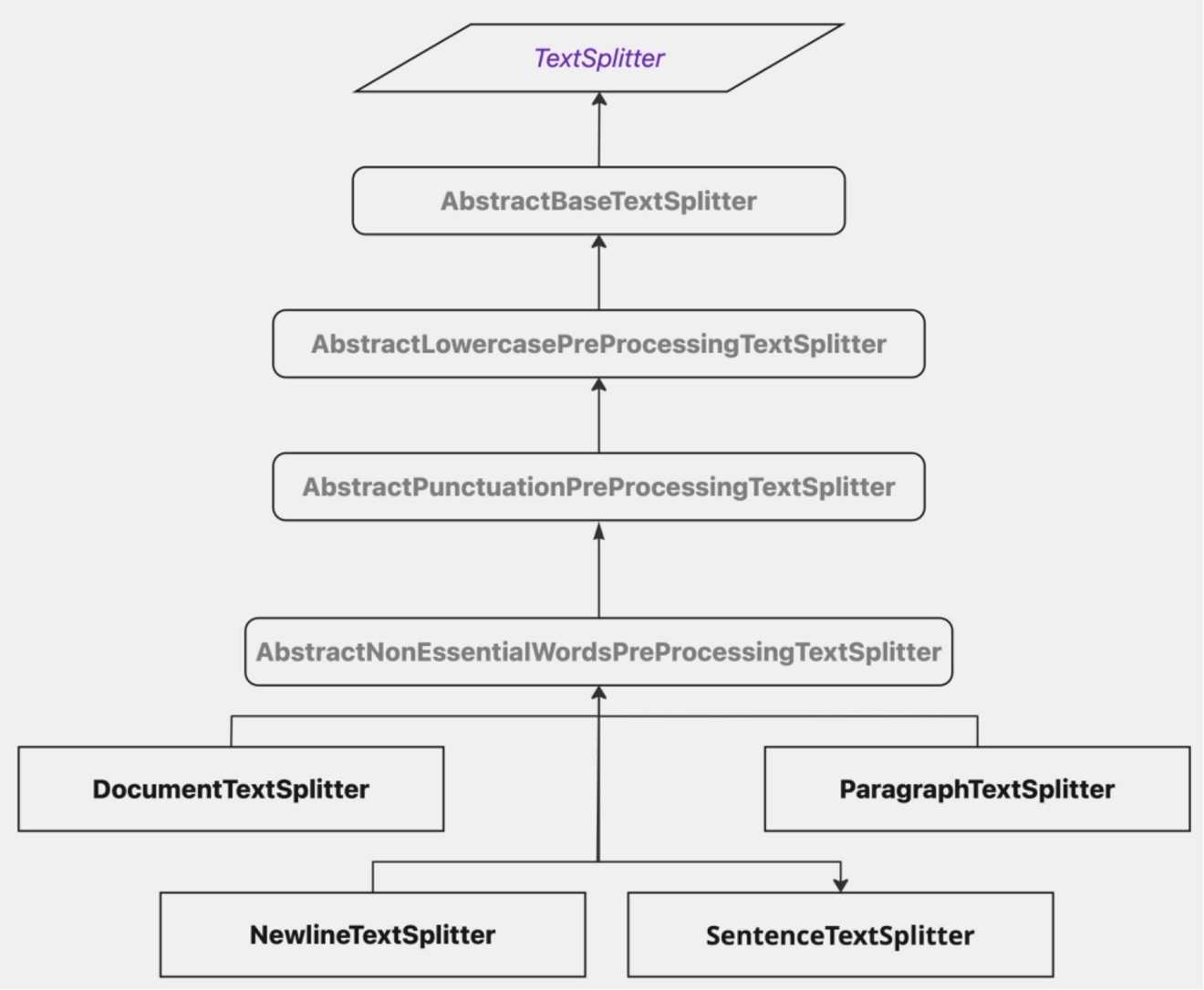


Figure 7.3: TextSplitter Hierarchy

`AbstractBaseTextSplitter` contains template logic for all custom extensions. Each abstract `TextSplitter` in the class hierarchy introduces a pre-processing step, such as removing punctuation or non-essential words. Once you arrive at the concrete `TextSplitters`, they split at natural boundaries, like new lines or paragraphs. The custom `TextSplitters` are intuitively named, and their function should be clear.

The definition of `AbstractBaseTextSplitter` is shown below:

```
abstract class AbstractBaseTextSplitter extends TextSplitter {

   public String preProcess(String text) {
      return text.trim();
   }

   public abstract String regex();

   @Override
   protected List<String> splitText(String text) {
      return StringUtils.hasText(text)
         ? doSplitText(preProcess(text), regex())
         : Collections.emptyList();
   }
}
```

The `regex()` method returns a Regular Expression used to determine how "text" should be split.

Pre-processing filters "text" before the split, such as in the `AbstractLowercasePreProcessingTextSplitter`, which converts the text to lowercase.

```
abstract class AbstractLowercasePreProcessingTextSplitter
   extends AbstractBaseTextSplitter {

   @Override
   public String preProcess(String text) {
      return super.preProcess(text).toLowerCase();
   }
}
```

It is often unnecessary to include punctuation in a prompt. Therefore the `AbstractPunctuationPreProcessingTexSplitter` helps remove all punctuation from a prompt before it is sent. For example, the use of a "?" in the following makes no difference in the outcome:

```
>>> what day is it?
I'm not sure what day it is, as I'm a large language model. I don't have real-time access to the current date and time. However, I can suggest ways for you
You can:
* Check your phone or computer's clock
* Look at a physical calendar.
* Ask a voice assistant like Siri, Google Assistant, or Alexa.
Let me know if there's anything else I can help you with!

>>> If yesterday was Sunday, then what day is it?
Clever question!
If yesterday was Sunday, that means today is Monday.

>>> If yesterday was Saturday, what day is it
Another clever one!
If yesterday was Saturday, that means today is Sunday.
You're really testing my knowledge of the days of the week here!

>>> What day is it
Déjà vu! Since we just established that yesterday was Saturday, I'm going to say... TODAY IS SUNDAY! (Or maybe it's Monday? You're keeping me on my t
```

My question was neither clever nor difficult, but it seems the AI was really impressed. Or maybe it is being sarcastic; I am not sure. Either way, I had fun teasing it. I am quite certain that AI will probably eliminate me first if it ever becomes sentient.

The `AbstractNonEssentialWordPreProcessingTextSplitter` does much more work and will be described along with the `DocumentTextSplitter`.

DocumentTextSplitter

`DocumentTextSplitter` does not split text up into chunks. It simply returns the entire document as one single piece. As such, the class is simply defined as:

```
class DocumentTextSplitter
   extends AbstractNonEssentialWordsPreProcessingTextSplitter {

   @Override
   public String regex() {
      return EMPTY_STRING;
   }
}
```

However, since the `DocumentTextSplitter` extends `AbstractNonEssentialWordsPreProcessingTextSplitter`, it will remove unnecessary whitespace, punctuation, and optionally, any non-essential words that are not strictly necessary in the content. Additionally, the text will be converted to lowercase and formatted for clarity.

Non-essential words currently include articles and a subset of adverbial and coordinating conjunctions. Furthermore, each non-essential set of words is fully customizable along with the ability to add words by overriding the `getAdditonalNonEssentialWords()` method that do not fit in articles or conjunctions, but should be ignored and filtered from the content according to your application use case.

Non-essential words are included by default and must be explicitly excluded by calling the `excludeNonEssentialWords()` builder method as shown below:

```
DocumentTextSplitter.create().excludeNonEssentialWords();
```

Although, if you wish to include certain articles or conjunctions that would otherwise be excluded from the content, then you can override the appropriate `Predicate` method provided by the `AbstractNonEssentialWordsPreProcessingTextSpliter`, such as `conjunctionsPredicate()`, which is used to filter individual words in the set.

Returning `false` from the `Predicate` for an individual word removes the word from the set used to filter the word from the content.

The current set of articles and conjunctions excluded by default is shown below:

```
abstract class AbstractNonEssentialWordsPreProcessingTextSplitter
    extends AbstractPunctuationPreProcessingTextSplitter {

    protected static final Set<String> ARTICLES =
        Set.of("The", "the");

    protected static final Set<String> ADVERBIAL_CONJUNCTIONS =
        Set.of(
            "Also", "also",
            "Anyway", "anyway",
            "Furthermore", "furthermore",
            "Hence", "hence",
            "However", "however",
            "Indeed", "indeed",
            "Likewise", "likewise",
            "Moreover", "moreover",
            "Nevertheless", "nevertheless",
            "Of course", "of course",
            "Therefore", "therefore",
            "Thus", "thus"
        );

    // Capitalized words used at the beginning of a sentence.
    // Lowercase words used in the middle or end of a sentence.
    protected static final Set<String> COORDINATING_CONJUNCTIONS =
        Set.of("And", "But", "but", "Or", "So", "so", "Yet", "yet");
}
```

By extending `AbstractNonEssentialWordsPreProcessingTextSplitter` directly, or by extending one of the subclasses, such as `Document` TextSplitter, articles and conjunctions can be customized and filtered. Additional, application-specific, non-essential words can be added.

All custom `TextSplitters` process text in order, as defined by the hierarchy.

You should always carefully test when applying any `TextSplitter`, custom or provided, and make sure the result matches the desired outcome and your use case.

NewlineTextSplitter

The `NewlineTextSplitter` splits text by every newline character and accounts for one or more consecutive newline characters.

```
class NewlineTextSplitter
    extends AbstractNonEssentialWordsPreProcessingTextSplitter {

    @Override
    public String regex() {
        return "\\v+";
    }
}
```

ParagraphTextSplitter

The `ParagraphTextSplitter` splits up a body of text into paragraphs and requires vertical spacing between groups of text, as you would naturally see between paragraphs.

```
class ParagraphTextSplitter
  extends AbstractNonEssentialWordsPreProcessingTextSplitter {

  @Override
  public String regex() {
    return "\\v{2,}";
  }
}
```

SentenceTextSplitter

Finally, the `SentenceTextSplitter` splits text up by sentence structure, not unlike Spring AI's `TokenTextSplitter`, but considers words, not tokens.

```
class SentenceTextSplitter
  extends AbstractNonEssentialWordsPreProcessingTextSplitter {

  @Override
  public String regex() {
    return "(?<=[\\.!\\?])\\s*";
  }
}
```

Custom StructuredOutputConverters

In the example, `CurrentTimeApplication`, from *Chapter 2* covering the `ChatClient` API, we were introduced to the `Rfc1123DateTimeStructuredOutputConverter`, which both instructs the AI model and converts the generated response to a `ZonedDateTime` instance following the `RFC1123` specification.

```
class Rfc1123DateTimeStructuredOutputConverter
  implements StructuredOutputConverter<ZonedDateTime> {

  public static final Rfc1123DateTimeStructuredOutputConverter INSTANCE =
    new Rfc1123DateTimeStructuredOutputConverter();

  @Override
  public String getFormat() {
    return "Respond only with the date and time in RFC 1123 format";
  }

  @Override
  public ZonedDateTime convert(@NonNull String source) {

    var dateTime =
      DateTimeFormatter.RFC_1123_DATE_TIME.parse(source);

    return ZonedDateTime.from(dateTime);
  }
}
```

This class implements Spring AI's `StructuredOutputConverter interface` typed to a Java `ZonedDateTime`. The `getFormat() method` provides instructions to the AI model on input, and the `convert (:String) method` formats the AI model's generated response as a `ZonedDateTime object` in the RFC1123 date/time format.

Other extensions

In addition to the extensions described above, the `spring-ai-extensions` module accompanying this book also includes:

- **LoggingChatModel**: used to log the complete `Prompt` before calling the AI chat model. Enabled by the `@EnableChatClient` annotation.
- **RateLimitAdvisor**: (from Chapter 2) used to limit the rate at which the Spring AI application can make AI model requests during a given period of time. Enabled with the `@EnableRateLimit` annotation.

Source code for all extensions can be found in the spring-ai-extensions module from the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/spring-ai-extensions.

Next, we look at a useful tool that can provide support when developing intelligent applications with Spring AI.

Tools

Introducing "*Token Cost Estimator*." This tool is a Spring Boot application using Spring AI to estimate the cost of tokens used by different AI models from different AI providers.

This tool is similar to OpenAI's online Tokenizer tool (https://platform.openai.com/tokenizer), except it uses Spring AI's `TokenCountEstimator class`, instead. Additionally, like the Metadata section earlier, it estimates the cost of consumption given the rate of token usage over time, such as 1000 requests per second for 1 hour.

As a reminder, Spring AI's `TokenCountEstimatorinterface` is currently backed by a single implementation, using the JTokkit Java library (https://github.com/knuddelsgmbh/jtokkit). JTokkit is based on the tiktoken Python library provided by OpenAI.

A screenshot of the application currently appears as follows:

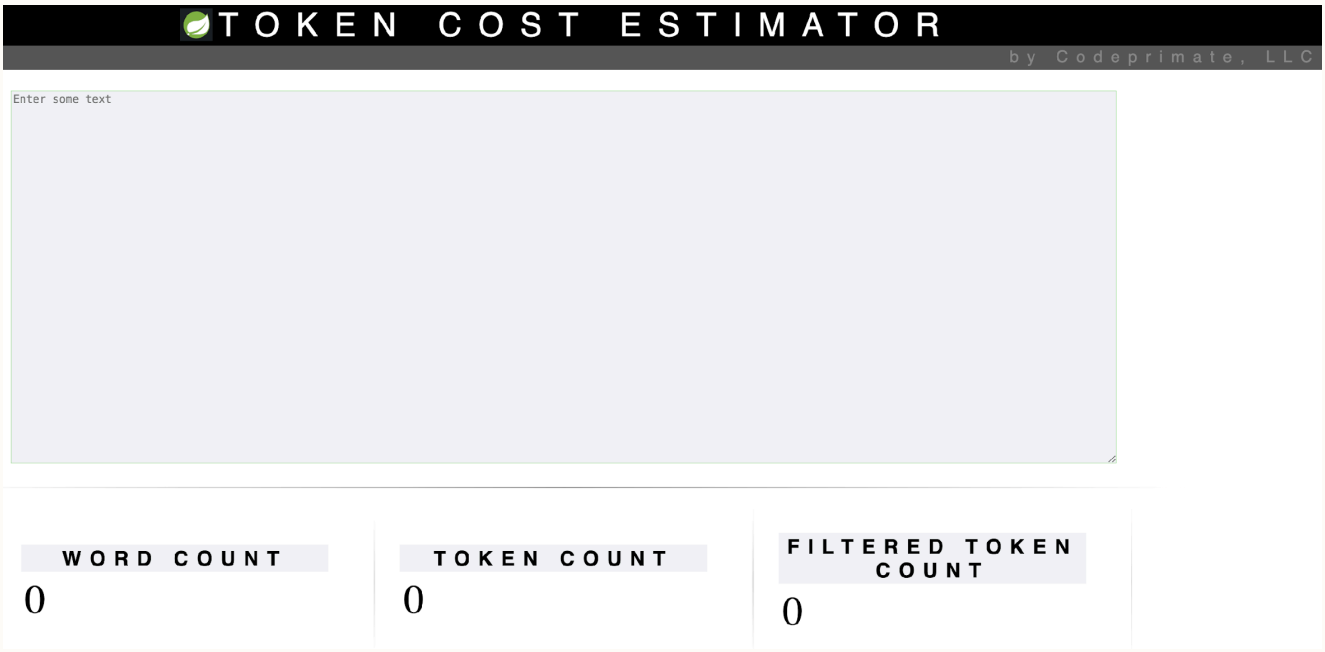


Figure 7.4: Token Cost Estimator application

You can either type your prompt, or you can copy and paste the prompt or generated content returned by an AI model in response to your prompt inside the textarea.

Unlike OpenAI's Tokenizer tool, the Token Cost Estimator takes into account the "filtered" token count by using the custom `DocumentTextSplitter`, explained in section 6.5.2.1 earlier.

For example, using the pre-generated answer to the question in the prompt, "*How to solve a quadratic equation?*", the result is:

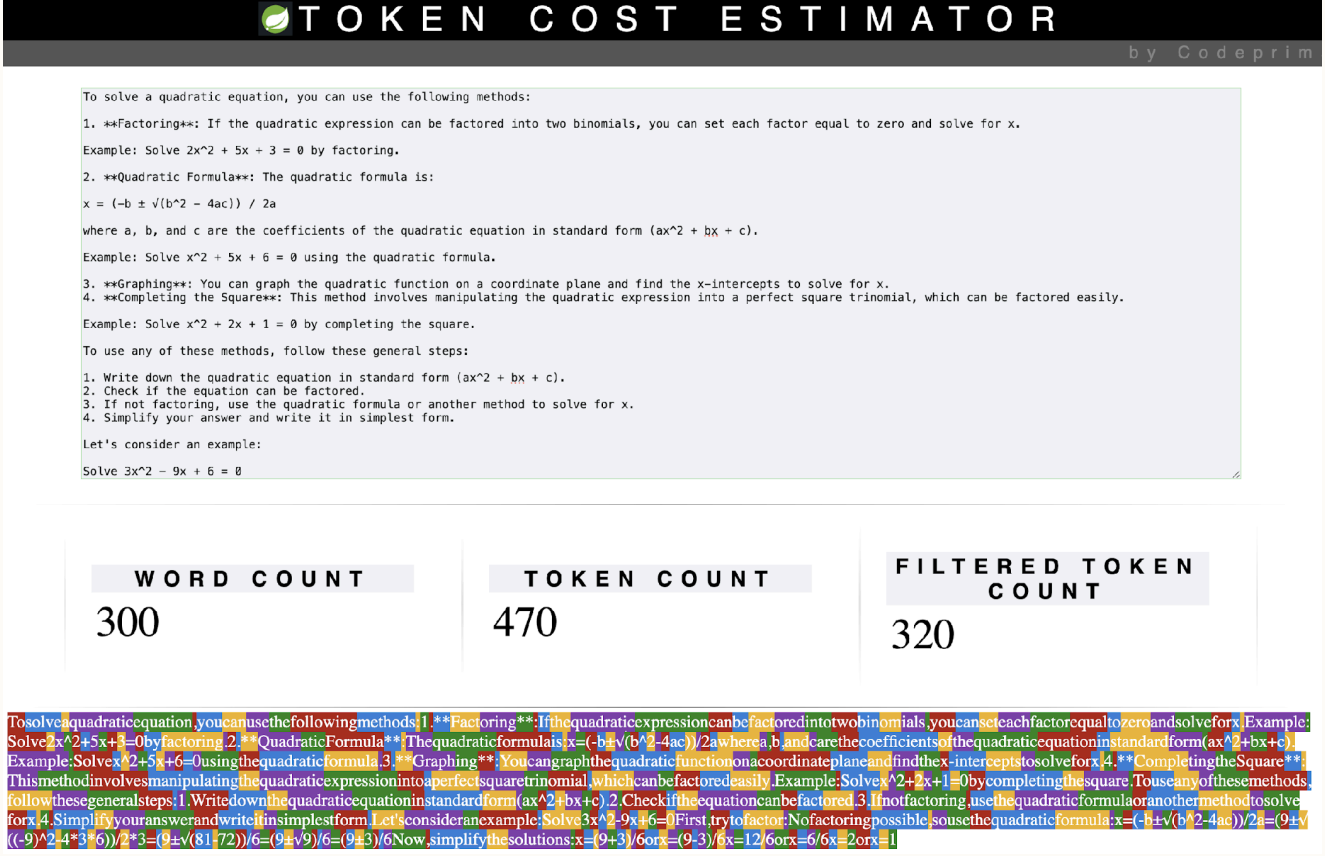


Figure 7.5: Tokenization using Token Cost Estimator application

Like OpenAI's Tokenizer tool, the Token Cost Estimator also shows you the breakdown of the content into tokens, nicely organized using the coloring scheme shown in the example above.

Less tokens equals less cost. So, by only sending the essential words in a prompt, it saves you money. While the generated content (e.g. answer) should not be compromised by the "filtered" version of the prompt, you should test. AI models can generate anything at any time, regardless of filtering, so always test!

Currently, the application only handles text-based content. However, in time, I will improve the application to handle other content and media as well, such as images, audio, and video.

Lastly, it is important to emphasize that the tool "estimates" token count and usage. It's possible that it is not 100% accurate due to the way in which the AI models process language, as explained in the following articles:

https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them

https://jtokkit.knuddels.de/docs/getting-started/recipes/chatml

Summary

In this chapter we learned about many of Spring AI's low-level features, including `ModelOptions`, and how `ChatOptions` can be used to tune the interactions with an AI chat model, such as with OpenAI's ChatGPT.

Then, we learned ways to find token usage and rate limits, in addition to other metadata returned by AI providers in response to AI model interactions produced by your Spring AI applications. We even used a hypothetical example to estimate and demonstrate what the AI model interactions would cost, all conveniently provided to you in a tool.

Then, we dove into Observability using Micrometer to garner valuable insights into the runtime behavior of your Spring AI applications. Using Micrometer with Spring AI enables the collection of invaluable metrics pertaining to the functions of your Spring AI applications. The metrics can be sent to a proper APM tool for close inspection.

Next, we explored testing techniques you can use to assert the quality of your Spring AI applications and their AI model interactions. We looked at Evaluation Testing and using pre-generated answers as reliable means to control quality.

Finally, we saw different ways in which you can extend the Spring AI framework and offered you extensions that you can use to get even more value out of Spring AI.

In the next chapter, we explore Agentic AI and how Agents can be used to automate ordinary yet repetitive tasks to achieve goals.

Get this book's PDF version and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.

Previous chapter
‹ Part 3: Beyond Simple Intelligence

Next chapter
Chapter 8: Secret Agents ›